
Métodos supervisados

PID_00296769

Jordi Casas Roma
Julià Minguillón Alfonso

Tiempo mínimo de dedicación recomendado: 3 horas



**Jordi Casas Roma**

Doctor en Informática (Universidad Autónoma de Barcelona, UAB, 2014), máster en Inteligencia Artificial Avanzada (Universidad Nacional de Educación a Distancia, UNED, 2011) y licenciado en Informática (Universidad Autónoma de Barcelona, UAB, 2002). Hasta 2023 ha sido profesor agregado de la Facultad de Informática, Multimedia y Telecomunicaciones de la Universitat Oberta de Catalunya (UOC). Desde 2024 es profesor lector del Departamento de Informática de la Universitat Autònoma de Barcelona (UAB). Sus actividades docentes se concentran principalmente en minería de datos, aprendizaje automático, *big data* y minería de gráficos. Sus principales ámbitos de investigación incluyen la inteligencia artificial, el aprendizaje automático, la minería de grafos y la privacidad de datos. Desde el 2020 es líder de grupo a ADaS Lab (Applied Data Science Lab) donde investiga la aplicación de algoritmos de inteligencia artificial a problemas del mundo real.

**Julià Minguillón Alfonso**

Profesor de la Universitat Oberta de Catalunya (UOC) desde 2001. Centra su actividad de investigación en el análisis y desarrollo de herramientas y recursos educativos abiertos en entornos virtuales de aprendizaje. Doctor por la Universitat Autònoma de Barcelona (UAB) con una tesis doctoral en el ámbito del aprendizaje automático. Sus líneas de trabajo se centran en el desarrollo de aplicaciones para la gestión, el análisis y la visualización de datos provenientes de entornos de aprendizaje virtuales, como el Campus Virtual de la UOC, y de redes sociales, especialmente Wikipedia. Es autor de diversos recursos docentes del ámbito de la ciencia de datos relacionados con la programación y el análisis y visualización de datos. Desde 2024 es el director del programa del máster universitario de Ciencia de Datos de la UOC.

Cómo citar este recurso de aprendizaje con el estilo Harvard:

Casas Roma, J.; Minguillón Alfonso, J. (2024). *Métodos supervisados*. [Recurso de aprendizaje textual]. 1.^a ed. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: septiembre 2024

© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoría: Jordi Casas Roma, Julià Minguillón Alfonso

Producción: FUOC

Todos los derechos reservados

Ninguna parte de esta publicación, incluido el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma, ni por ningún medio, sea este eléctrico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita del titular de los derechos.

Pasados tres años desde la fecha de publicación de la primera edición de este material, se considerará publicado bajo licencia Creative Commons (https://creativecommons.org/licenses/by-nc-nd/4.0/deed.es_ES).



Índice general

Introducción	5
Objetivos	6
1 Contextualización	7
2 Algoritmo k-NN	8
2.1 Funcionamiento del método	8
2.2 Detalles del método	9
2.3 Ejemplo de selección empírica del valor de k	11
2.4 Resumen	12
3 Árboles de decisión	13
3.1 Detalles del método	16
3.2 Poda del árbol	21
3.3 Resumen	23
4 Máquinas de soporte vectorial	25
4.1 Definición de margen e hiperplano separador	26
4.2 Cálculo del margen y del hiperplano separador óptimo	27
4.3 Funciones <i>kernel</i>	30
4.4 Tipos de funciones <i>kernel</i>	35
4.5 Resumen	38
Resumen	39
Glosario	40
Bibliografía	41

Introducción

El aprendizaje automático supervisado tiene como objetivo principal aprender los patrones y relaciones en datos a través de la observación de ejemplos etiquetados. Esta técnica se basa en la utilización de algoritmos y modelos matemáticos que permiten realizar predicciones a partir de datos previamente etiquetados con una clase que es la predicción a realizar.

El término «supervisado» implica que durante el proceso de entrenamiento, el algoritmo se ajusta mediante un conjunto de datos de entrenamiento, en el cual cada muestra contiene una entrada y una etiqueta objetivo o clase correspondiente. Estas etiquetas representan las respuestas deseadas, y el objetivo del modelo es aprender a generalizar y hacer predicciones precisas para nuevos datos no vistos con anterioridad.

Los modelos supervisados tienen aplicaciones en una amplia variedad de campos, como el procesamiento de lenguaje natural, la visión por computador, el diagnóstico médico, la detección de fraudes y la recomendación de productos, entre muchos otros. Las capacidades para extraer patrones complejos y aprender a partir de datos han convertido a los modelos supervisados, probablemente, en el tipo de aprendizaje automático más popular en la actualidad.

Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Comprender las características y principales aplicaciones del aprendizaje supervisado, así como sus requerimientos y limitaciones.
2. Conocer el método k -NN y sus principales características.
3. Conocer los árboles de decisión, su funcionamiento y parámetros, y sus principales características.
4. Conocer las máquinas de soporte vectorial (SVM), su funcionamiento y parámetros, y sus principales características.

1. Contextualización

El principal objetivo de los modelos supervisados de aprendizaje automático es realizar predicciones precisas y generalizadas a partir de datos previamente etiquetados. Estos modelos buscan aprender patrones y relaciones en los datos de entrenamiento para poder hacer inferencias sobre datos nuevos y no vistos previamente.

En un modelo supervisado, durante el proceso de entrenamiento, se proporciona al algoritmo un conjunto de datos de entrenamiento que contiene ejemplos con sus respectivas etiquetas o resultados deseados. El modelo utiliza esta información para ajustar sus parámetros y encontrar la mejor representación de las relaciones entre las características de entrada (variables independientes) y las etiquetas objetivo o clases (variable dependiente).

En relación con la notación de este texto, emplearemos la siguiente notación para referirnos al conjunto de entrada de datos:

$$D_{n,m} = \begin{pmatrix} d_1 = & a_{1,1} & a_{1,2} & \cdots & a_{1,m} & c_1 \\ d_2 = & a_{2,1} & a_{2,2} & \cdots & a_{2,m} & c_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ d_n = & a_{n,1} & a_{n,2} & \cdots & a_{n,m} & c_n \end{pmatrix}$$

donde:

- n es el número de elementos o instancias,
- m es el número de atributos del conjunto de datos o también su dimensionalidad,
- d_i indica cada una de las instancias del juego de datos,
- $a_{i,j}$ indica cada uno de los atributos descriptivos, y
- c_i indica el atributo objetivo o clase a predecir para cada elemento. Suponemos que cada elemento pertenece solamente a una clase.

Es importante destacar que el atributo objetivo o clase a predecir (c_i) es imprescindible en los métodos supervisados. Es la «verdad» que deseamos que el algoritmo aprenda a partir de los datos.

2. Algoritmo k -NN

El k -NN o k vecinos más cercanos (en inglés, *k nearest neighbours*) es un algoritmo de aprendizaje supervisado de clasificación, de modo que, a partir de un juego de datos de entrenamiento, su objetivo será clasificar correctamente todas las instancias nuevas. El juego de datos típico de este tipo de algoritmos está formado por varios atributos descriptivos y un solo atributo objetivo, también llamado *clase*.

2.1 Funcionamiento del método

En contraste con otros algoritmos de aprendizaje supervisado, k -NN no genera un modelo fruto del aprendizaje con datos de entrenamiento, sino que el aprendizaje sucede en el mismo momento en el que se pide clasificar una nueva instancia. A este tipo de algoritmos se les llama algoritmos de aprendizaje perezoso o *lazy learning methods*, en inglés.

El funcionamiento del algoritmo es muy simple. Para cada nueva instancia a clasificar, se calcula la distancia con todas las instancias de entrenamiento y se seleccionan las k instancias más cercanas. La clase de la nueva instancia se determina como la clase mayoritaria de sus k instancias más cercanas. La idea es que si un elemento está rodeado (en el espacio multidimensional) mayoritariamente por elementos de una clase, entonces es más probable que dicho elemento sea de dicha clase.

El funcionamiento del método se detalla en el algoritmo 1 y se describe a continuación:

- 1) Fijamos un valor para k , habitualmente pequeño.
- 2) Dada una nueva instancia x del juego de datos de prueba, el algoritmo selecciona las k instancias del juego de datos de entrenamiento más cercanas, de acuerdo con la métrica de similitud utilizada.
- 3) Finalmente, se asigna la instancia x a la clase más frecuente de entre las k instancias seleccionadas como más cercanas, resolviendo los posibles empates.

Sin duda se trata de un algoritmo tremendamente simple y, a la par, con un nivel de efectividad similar a otros algoritmos más complejos y elaborados,

especialmente si los datos tienen una estructura interna ligada al espacio multidimensional en el cual están representados.

Algoritmo 1 Pseudocódigo del algoritmo k -NN

Require: k (número de vecinos) y x (nueva instancia a clasificar)

for all $d_i \in D$ **do**

 Calcular la distancia $d(d_i, x)$

end for

 Inicializar I con las clases c_i de las k instancias de entrenamiento más próximas a x

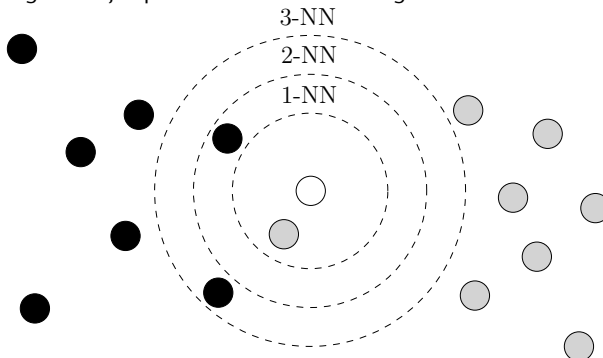
return La clase mayoritaria en I

Veamos gráficamente, mediante la figura 1, las distintas clasificaciones que se obtienen al variar el valor de k . Supongamos que debemos clasificar una nueva instancia, representada aquí como una bola blanca, utilizando un conjunto de datos de entrenamiento binario, representado por bolas de color gris o negro.

Según el valor de k , la nueva instancia será clasificada como:

- Para $k = 1$ el algoritmo clasificará la nueva instancia como clase «gris».
- Para $k = 2$ el algoritmo no tiene criterio para clasificar la nueva instancia.
- Para $k = 3$ el algoritmo clasificará la nueva instancia como clase «negra».

Figura 1. Ejemplo de valores de k en el algoritmo k -NN



Su mayor debilidad es la lentitud en el proceso de clasificación, puesto que su objetivo no es obtener un modelo optimizado, sino que cada instancia de prueba es comparada contra todo el juego de datos de entrenamiento. Será la bondad de los resultados lo que determinará el ajuste de aspectos del algoritmo como el propio valor k , el criterio de selección de instancias para formar parte del juego de datos D de entrenamiento o la propia métrica de medida de similitud.

2.2 Detalles del método

Este método presenta dos variables importantes que pueden determinar la bondad del método. Estas son la métrica de similitud escogida y el valor de la propia k .

La métrica de similitud utilizada debería tener en cuenta la importancia relativa de cada atributo, puesto que esta influirá fuertemente en las relaciones de cercanía que se irán estableciendo en el proceso de construcción del algoritmo. La métrica de distancia puede llegar a contener pesos que nos ayudarán a calibrar el algoritmo de clasificación, convirtiéndola de hecho en una métrica personalizada. Además, debería ser eficiente computacionalmente, ya que deberemos de ejecutar el cálculo de similitud multitud de veces durante el proceso de clasificación de nuevas instancias.

Una de las distancias más utilizadas es la euclídea, especialmente en el caso de tratar con atributos numéricos:

$$de(x, d_i) = \sqrt{\sum_{j=1}^m (x_j - d_{ij})^2} \quad (1)$$

En el caso de tratar con atributos nominales o binarios, una de las más utilizadas es la distancia de Hamming:

$$dh(x, d_i) = \sum_{j=1}^m \delta(x_j, d_{ij}) \quad (2)$$

donde $\delta(x_j, d_{ij})$ toma el valor 0 si $x_j = d_{ij}$ y 1 en caso contrario.

El valor de la variable k es muy importante en la ejecución de este método, de modo que con valores distintos de k podemos obtener resultados también muy distintos. Este valor suele fijarse tras un proceso de pruebas con varias instancias. Se suele escoger un número impar para minimizar la posibilidad de empates en el momento de decidir la clase de una nueva instancia. Aun así, cuando se produce un empate se debe decidir cómo clasificar la instancia. Algunas alternativas pueden ser, por ejemplo, no dar predicción o dar la clase más frecuente en el conjunto de aprendizaje de las clases que han generado el empate.

Otro factor importante que se debe considerar antes de aplicar el algoritmo k -NN es el rango u orden de los datos. Es importante expresar los distintos atributos en valores que sean «comparables». Por ejemplo, si modificamos un atributo que originalmente estaba expresado en kilogramos y lo transformamos en gramos, el resultado de las métricas de distancias pueden cambiar drásticamente. En este sentido, existen transformaciones, como por ejemplo la unidad tipificada (*standard score* o *z-score* en inglés) que resta a cada valor el valor medio del atributo y lo divide por su respectiva desviación estándar.

Merece la pena comentar que puede llegar a producirse el efecto del sobreentrenamiento del modelo (*overfitting*). El sobreentrenamiento provoca que el modelo que se construye, en lugar de describir las estructuras subyacentes del

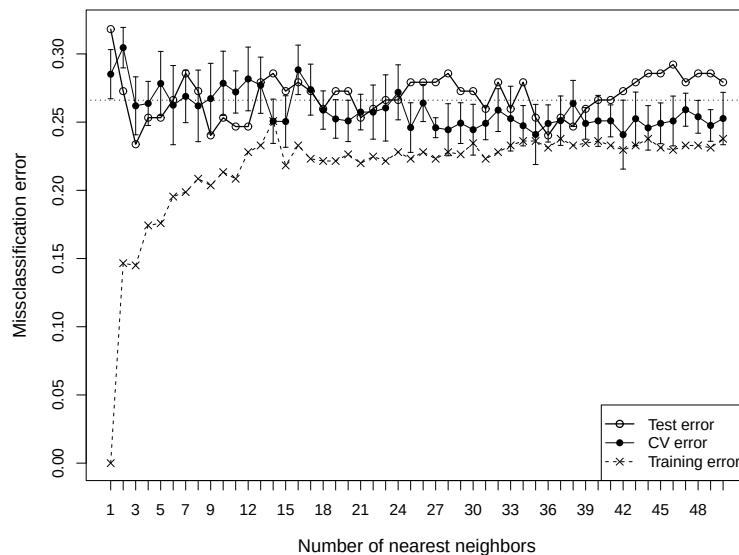
juego de datos, lo que acaba describiendo sea exclusivamente el juego de datos de entrenamiento, no siendo extrapolable a otros juegos de datos nunca vistos con anterioridad.

2.3 Ejemplo de selección empírica del valor de k

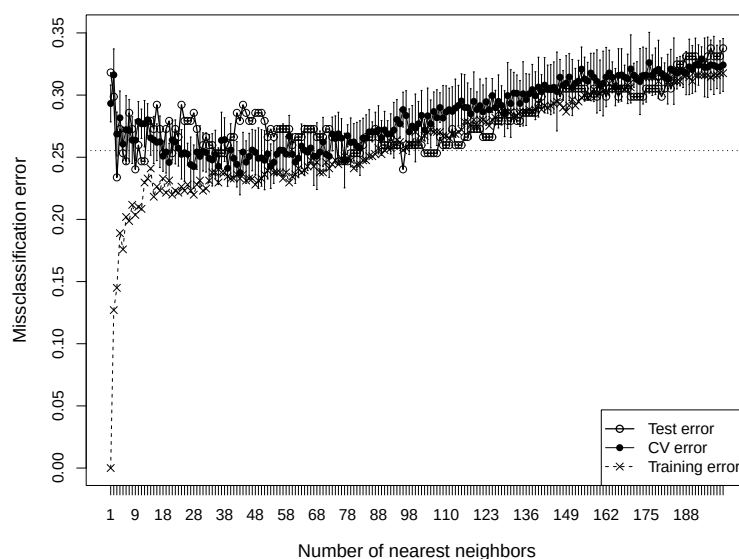
Veamos un ejemplo de cómo seleccionar el valor de la k utilizando el conjunto de datos PimaIndiansDiabetes¹. Este conjunto de datos presenta 768 instancias y 9 atributos utilizados para determinar si un paciente tiene diabetes. El 35% de las instancias pertenece a la clase de pacientes positivos, es decir, que finalmente se les diagnosticó diabetes.

⁽¹⁾<https://archive.ics.uci.edu/ml/datasets/Pima+Indians+Diabetes>

Figura 2. Evolución del error en función del parámetro k



(a) Valores de k en el rango $\{1, \dots, 50\}$



(b) Valores de k en el rango $\{1, \dots, 200\}$

La figura 2a muestra los resultados de un conjunto de pruebas con valores de $k = \{1, \dots, 50\}$ (eje horizontal) utilizando el método *10-fold cross-validation*. La figura muestra tres tipos de errores (eje vertical), que se corresponden con el error del conjunto de test (*test error*), el error en las pruebas de validación cruzada (*CV error*) y el error en el conjunto de entrenamiento (*training error*). Como se puede apreciar, a partir de un valor de $k = 15$, el error de test y de validación cruzada no experimentan cambios importantes. Por lo tanto, un valor entre 18 y 26 parece adecuado. Es interesante notar que el error de entrenamiento es muy bajo para valores de $1 \leq k \leq 5$, pero considerablemente por encima de la media en el caso de considerar un conjunto de test o una estimación mediante validación cruzada.

Cuando el valor del parámetro k aumenta demasiado, aparte del coste computacional, estaremos considerando instancias próximas y no tan próximas para determinar la clase de un individuo concreto, lo que deriva en problemas de subentrenamiento (*underfitting*). La figura 2b muestra un proceso similar al visto en la figura 2a, pero donde analizamos el comportamiento del algoritmo en un intervalo de $k = \{1, \dots, 200\}$. Se puede apreciar claramente como el comportamiento del algoritmo se degrada de forma considerable para un valor de $k \geq 70$.

2.4 Resumen

Los principales puntos fuertes de este método de clasificación supervisado son la simplicidad y la robustez ante el ruido presente en los datos de entrenamiento. Además, el hecho de «crear» específicamente un modelo para procesos de predicción puede ser visto como una ventaja importante en ciertos problemas o dominios.

Por el contrario, algunas de las principales limitaciones de este método también están relacionadas con el hecho de que no se cree un modelo específico durante el entrenamiento. Esto provoca que cada nueva instancia deba ser comparada con las demás para encontrar los k vecinos más próximos e identificar la clase de la nueva instancia. Por lo tanto, el proceso de clasificación de cada nueva instancia no es trivial a nivel de coste computacional, y crece al aumentar el conjunto de entrenamiento etiquetado.

En este sentido, es muy importante escoger una implementación del algoritmo que permita una ejecución óptima de cálculo y memoria. Además, el rendimiento se puede ver seriamente afectado cuando se trata con conjuntos con muchos atributos, como por ejemplo en el caso de imágenes que pueden presentar cientos o miles de dimensiones, problema que se conoce como la «maldición de la dimensión» (*the curse of dimensionality*).

3. Árboles de decisión

Los árboles de decisión son uno de los modelos de minería de datos más comunes y estudiados, y no precisamente por su capacidad predictiva, superada generalmente por otros modelos más complejos, sino por su alta capacidad explicativa y la facilidad para interpretar el modelo generado. Por ejemplo, son ampliamente utilizados en sectores como el bancario y las compañías aseguradoras para tomar decisiones respecto a la concesión de créditos o el cálculo de las pólizas, respectivamente, dado que permiten determinar qué características relativas a los usuarios son las que comportan mayor o menor riesgo, siendo posible segmentar a los usuarios en función de dichas características.

Como modelos supervisados que son, a los árboles de decisión que clasifican los datos del conjunto de entrada en función de una variable clasificadora categórica (es decir, que toma un conjunto finito de valores) se les llama árboles de clasificación. Si la variable clasificadora es continua, hablaríamos de árboles de regresión. Ambos tipos de árboles de decisión dan el nombre común CART (*Classification and Regression Trees*) según el trabajo original descrito por Breiman, Friedman, Stone y Olshen en 1984 (Breiman *et al.*, 1984).

Básicamente, los árboles de decisión son un modelo de minería de datos que intenta subdividir el espacio de datos de entrada para generar regiones disjuntas, de forma que todos los elementos que pertenezcan a una misma región sean de la misma clase, la cual es utilizada como representante o clase de dicha región. Si una región contiene datos de diferentes clases, es subdividida en regiones más pequeñas siguiendo el mismo criterio, hasta particionar todo el espacio de entrada en regiones disjuntas que solamente contienen elementos de una misma clase. Así, un árbol de decisión se llama completo o puro si es posible generar una partición del espacio donde cada subregión solo contenga elementos de una misma clase. Esto siempre será posible, excepto si en el conjunto de datos de entrada existen elementos idénticos etiquetados con clases diferentes. En este caso será necesario decidir si se trata de *outliers* o bien se trata de casos que deben ser tratados de forma separada.

Respecto a su estructura, un árbol de decisión consta de nodos hoja o terminales, que representan regiones etiquetadas o clasificadas de acuerdo a una clase, y nodos internos o *splits* que representan condiciones que permiten decidir a qué subregión va cada elemento que llega a dicho nodo, es decir, la partición realizada. Cuando se presenta un nuevo dato a un árbol de decisión, se empieza por el nodo raíz que contiene una condición que determina por qué rama del árbol debe descenderse (es decir, a qué subregión pertenece el dato)

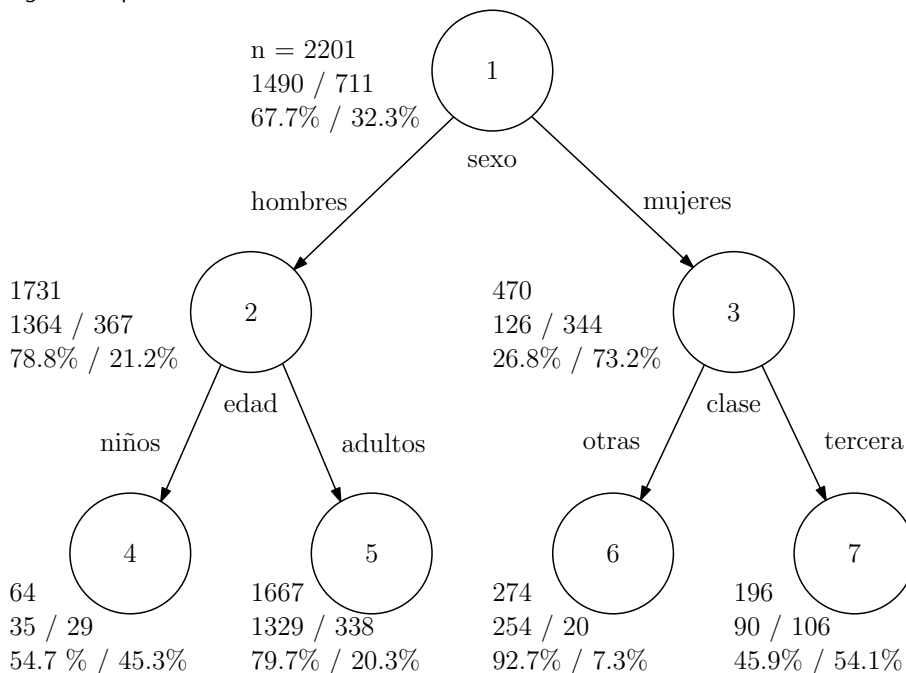
Lectura complementaria

Leo Breiman, Jerome Friedman, Charles J. Stone, R. A. Olshen, (1984). «Classification and Regression Trees», Chapman and Hall/CRC

hasta alcanzar la siguiente condición o bien una hoja terminal, en cuyo caso se determina qué clase corresponde al nuevo dato. Contrariamente a la idea intuitiva ligada al concepto de «árbol», la raíz es el elemento superior y las hojas se sitúan en posiciones más profundas. Así, la profundidad máxima de un árbol de decisión es el máximo número de condiciones que es necesario resolver para llegar a una hoja, siendo posible medir también la profundidad media, ponderando la profundidad de cada hoja con respecto al número de elementos del conjunto de entrenamiento que contiene.

Por lo tanto, un árbol de decisión es una secuencia de condiciones que son interrogadas con respecto a los datos de entrada, tomando una decisión parcial que lleva hacia una rama u otra, repitiendo este proceso hasta llegar a una hoja donde se toma una decisión final. Por ejemplo, en el hundimiento del Titanic, mostrado en la figura 3, de las 2201 personas a bordo solamente sobrevivieron 711 (un 32.3%), mientras que las 1490 restantes fallecieron.

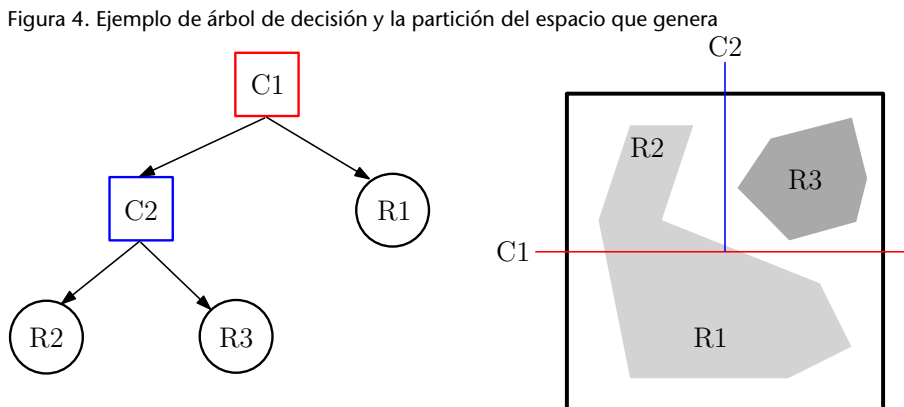
Figura 3. Supervivencia de los embarcados en el Titanic



Si se tuviera que tomar una decisión sobre todos los embarcados, esta sería «No sobrevivieron», dado que es correcta en el 67.7% de los casos. Pero si analizamos la supervivencia con respecto a otras variables disponibles, la primera condición es el sexo, dado que las mujeres a bordo del Titanic (344 de 470, un 73.2%) sobrevivieron en mayor proporción que los hombres (367 de 1731, un 21.2%). Esto genera una partición del conjunto de entrada en dos conjuntos disjuntos; tras la primera condición (el nodo raíz), por una rama del árbol (izquierda) solo quedarán hombres y por la otra (derecha) mujeres. La decisión con respecto a las mujeres sería «Sobrevivieron», dado que lo hicieron en un 73.2%. En cambio, en el caso de los hombres, la decisión sería «No sobrevivieron», la cual sería cierta en un 78.8% de los casos. Se puede observar que

la precisión de la predicción ha aumentado para ambas hojas con respecto a la primera predicción. Esto siempre sucede al menos para una hoja, dado que el procedimiento seguido intenta mejorar la predicción anterior. De la misma manera, en el caso de los hombres, la siguiente condición es la edad, dado que los niños sobrevivieron en una proporción mayor que los adultos. En cambio, en el caso de las mujeres, la siguiente condición es la clase en la cual viajaban, dado que las mujeres que lo hacían en primera clase sobrevivieron más que las que lo hacían en segunda y mucho más que las que lo hacían en tercera clase. De hecho, las mujeres de tercera clase tuvieron una tasa de supervivencia inferior al 50%, mientras que el resto sobrevivió en más de un 90%. Nótese que cada rama puede contener una secuencia de condiciones diferentes, lo cual aporta información sobre el problema a resolver. De hecho, este árbol de decisión representa razonablemente bien la típica frase «las mujeres (no pobres) y los niños primero» usada en catástrofes de este tipo.

La figura 4 muestra un ejemplo de árbol de decisión y una representación de la partición del espacio que genera, intentando separar elementos de dos clases diferentes (representadas por dos tonos de gris) en un espacio de datos de entrada de dos dimensiones. En este caso, la profundidad máxima del árbol es 2. La primera condición (C1) crea dos regiones, una de las cuales (R1) solamente contiene elementos de una clase (todos de gris claro), por lo que no es necesario continuar dividiéndola. La otra región es dividida de acuerdo a una segunda condición (C2), creando dos nuevas regiones (R2 y R3), las cuales contienen solamente elementos de la misma clase.



El algoritmo genérico para construir un árbol de decisión utiliza los siguientes criterios:

- C_p o criterio de parada, determina en qué momento se deja de seguir seleccionando nodos para ser subdivididos. El más habitual es generar un árbol completo, lo cual quiere decir que no quedan nodos que particionar.
- C_s o criterio de selección, determina qué nodo es seleccionado para ser particionado en dos o más subnodos. Existen diferentes criterios de selección,

pero si se generan árboles de decisión completos, el criterio utilizado no es relevante, dado que tarde o temprano todos los nodos serán candidatos a ser particionados.

- C_c o criterio de clasificación, determina qué clase se asigna a un nodo hoja. Normalmente se trata de la clase que minimiza el error de clasificación o el valor numérico que minimiza el error de regresión.
- C_d o criterio de partición, determina cómo se particiona un nodo en dos o más subnodos. Normalmente los árboles de decisión son binarios (es decir, $P = 2$), indicando que cada región se subdivide en dos regiones disjuntas. Esto simplifica el criterio de partición y mejora la interpretabilidad del árbol a costa de generar árboles más profundos.

El algoritmo genérico para construir un árbol de decisión, o lo que es equivalente, una partición del espacio de entrada es el siguiente:

Algoritmo 2 Pseudocódigo para construir un árbol de decisión

Require: Conjunto de datos a clasificar D

$T = \{D\}$ // El árbol inicial es un solo nodo hoja

Etiquetar T de acuerdo a C_c

$p = \{T\}$ // Lista de nodos pendientes (hojas)

while no se deba parar según C_p **do**

 Seleccionar un nodo q de acuerdo a C_s

if es posible particionar q según C_d **then**

 Particionar q en $q_1, \dots, q_p$

 Etiquetar $q_1, \dots, q_p$ según C_c

 Añadir $q_1, \dots, q_p$ a p

 Sustituir q en T por un nodo interno

end if

 Eliminar q de p

end while

return T

Los dos criterios más importantes son los de clasificación (C_c) y el de partición (C_d), dado que determinan el árbol construido a partir del conjunto de datos de entrada, especialmente cuando se construyen árboles completos.

3.1 Detalles del método

Como ya se ha comentado, el árbol de decisión construido depende de los cuatro criterios anteriormente mencionados: parada, selección, clasificación y partición. Los cuatro criterios están relacionados entre sí y pueden ser establecidos independientemente.

3.1.1 El criterio de parada

El criterio de parada determina cuándo se dejan de subdividir nodos para decidir que ya se ha construido un árbol suficientemente adecuado para los datos de entrada a clasificar. Este criterio suele usar aspectos relativos a la estructura del árbol y a su rendimiento como clasificador. Por ejemplo, puede decidirse parar cuando el árbol alcanza una cierta profundidad máxima o bien cuando la profundidad media (ponderada para todas las hojas) alcanza cierto valor. Otra posibilidad es medir el error cometido para los datos de entrada y parar cuando dicho error está por debajo de un cierto nivel. Obviamente, si no hay ninguna hoja que pueda ser particionada, el algoritmo también se detiene. En este sentido, si el objetivo es construir un árbol completo, el criterio de parada es simplemente no parar mientras exista alguna hoja que pueda ser particionada.

3.1.2 El criterio de selección

El criterio de selección determina qué nodo hoja es escogido para ser particionado. Obviamente, una hoja que solamente contenga elementos de una misma clase no debe ser particionada, ya que no mejora la capacidad predictiva del árbol, así que este criterio combina aspectos relativos a la impureza de las hojas, combinando el error cometido en dicha hoja y otras características como el número de elementos de conjunto de entrada que contiene o su profundidad en el árbol. Si el objetivo es construir un árbol completo, todas las hojas impuras deberán ser particionadas, sea en el orden que sea, por lo que el criterio puede ser tan sencillo como seleccionar el siguiente nodo en la lista de nodos pendientes. No obstante, el criterio habitual es seleccionar el nodo más impuro, es decir, aquel que contiene una mayor mezcla de elementos de diferentes clases. Habitualmente esto puede medirse mediante la entropía, que mide el grado de desorden de una distribución de elementos de k clases diferentes:

$$H = \sum_{i=1}^k p_i \log p_i \quad (3)$$

La entropía es cero si todos los elementos son de una misma clase c , ya que entonces $p_c = 1$ y el resto de $p_i = 0$ ($\forall i \neq c$) (se asume que $0 \log 0 = 0$). Cada p_i se calcula como el número de elementos de la clase i presentes en la hoja t dividido por el total de elementos en dicha hoja n_t , de la manera siguiente:

$$p_i(t) = n_i(t)/n_t \quad (4)$$

Es importante comentar que no se debe confundir esta k (número de clases diferentes que aparecen en los diferentes $c_1, \dots, c_n$ que componen el conjunto de datos D) con el parámetro k del algoritmo k -NN.

3.1.3 El criterio de clasificación

El criterio de clasificación determina qué clase se asigna a una región u hoja. Este criterio determina el error cometido en aquella hoja y también el error global que comete el árbol de decisión. Obviamente, si una hoja solamente contiene elementos de una clase, dicha clase es la elegida como representante de la región, minimizando el error cometido en la hoja, el cual es cero. En el caso de que en una hoja existan elementos de diferentes clases, se escogerá aquella clase que minimiza el error cometido, normalmente la más poblada. En el improbable caso de empate entre diferentes clases, puede tomarse una al azar, o el de la clase más popular en el conjunto de datos original. En general, la clase escogida es aquella que satisface la siguiente ecuación:

$$c(t) = \arg \min_j \sum_{i=1}^k p_i(t) C_{i,j} \quad (5)$$

Es decir, se trata de escoger aquella clase que minimiza el error cometido, teniendo en cuenta el posible diferente coste $C_{i,j}$ de cometer un error al escoger una clase i en lugar de la otra j , es decir, con costes asimétricos ($C_{i,j} \neq C_{j,i}$). Obviamente, $C_{i,i} = 0 \forall i$, es decir, el coste de clasificar un elemento de una clase por dicha clase es 0, dado que no cometemos ningún error. Así, por ejemplo, en un árbol de decisión que intenta determinar mediante la resonancia magnética de una muestra de tejido si esta se trata de un tumor o no, puede resultar interesante cometer un error y obtener falsos positivos (es decir, indicar que hay un tumor cuando no es así) en lugar de falsos negativos (es decir, indicar que no se trata de un tumor cuando realmente sí lo es), por lo que los costes se pueden ajustar para alcanzar este objetivo.

El error total cometido por un árbol T es la suma ponderada del error cometido en cada hoja, teniendo en cuenta el tamaño de la región definido por cada hoja, estimado a través del número de elementos que la componen:

$$e(T) = \frac{1}{n} \sum_{t \in T} n_t \sum_{i=1}^k p_i(t) C_{i,c(t)} \quad (6)$$

que es equivalente a:

$$e(T) = \frac{1}{n} \sum_{t \in T} \sum_{i=1}^k n_i(t) C_{i,c(t)} \quad (7)$$

3.1.4 El criterio de partición

El criterio de partición determina cómo se divide una hoja en dos o más regiones. De hecho, el número de regiones y del criterio utilizado determinan qué tipo de árbol de decisión se está construyendo, dado que existen diferentes algoritmos que intentan aprovechar la naturaleza de los datos de entrada. Lo habitual es trabajar con árboles binarios, donde la región a dividir se particiona en dos regiones disjuntas. Se puede pensar como si un hiperplano cortara el espacio de entrada en dos. En el ejemplo de la figura 4, cada una de las condiciones C1 y C2 representa este hiperplano.

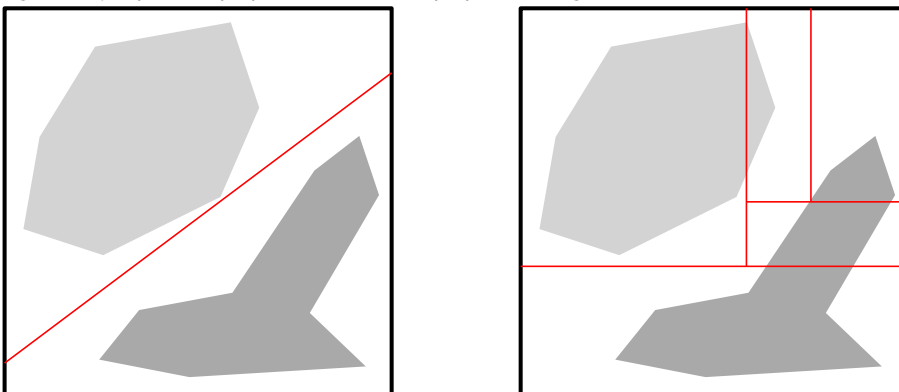
El criterio de partición combina, de hecho, dos criterios diferentes: uno determina cómo se construye el hiperplano y el otro determina en qué posición del espacio se sitúa el hiperplano para realizar la partición.

En el primer caso, lo más habitual es utilizar hiperplanos ortogonales a cada uno de los ejes (variables) del espacio de entrada. Es decir, cada hiperplano es, en realidad, una condición que particiona los elementos de una hoja en dos subconjuntos en función de un valor dado. Es decir, en cada nodo interno solo se utiliza una variable para decidir la clasificación de un elemento. Otra opción es buscar un hiperplano oblicuo que seguramente mejorará los resultados obtenidos con el método anterior, aunque la obtención de dicho hiperplano puede ser muy costosa, complicará su interpretación posterior y, además, puede provocar problemas de sobreentrenamiento y mala generalización para datos nunca vistos. En el caso mostrado en la figura 5, un único hiperplano oblicuo puede separar perfectamente las dos clases presentes en el conjunto de datos, mientras que si se usaran hiperplanos ortogonales serían necesarios al menos cuatro, intentando reproducir el contorno de cada una de las regiones. El lector interesado en la construcción de hiperplanos oblicuos puede consultar el trabajo de Murthy *et al.*, 1994.

Lectura complementaria

Sreerama K. Murthy, Simon Kasif, Steven Salzberg. (1994). «A system for induction of oblique decision trees». *J. Artif. Int. Res.* 2, 1 (August 1994), 1-32. <http://dl.acm.org/citation.cfm?id=1622826.1622827>

Figura 5. Ejemplo de hiperplano oblicuo vs. hiperplanos ortogonales



En el segundo caso, para determinar la posición del hiperplano o corte para todos los hiperplanos posibles, lo que se hace es medir como mejoraría el

árbol si se aplicara la partición definida por cada hiperplano, maximizando o minimizando algún criterio. Una opción que parece *a priori* interesante es utilizar el hiperplano que maximiza la reducción del error cometido por el árbol de decisión, de la forma siguiente: sea T el árbol antes de realizar la partición de una hoja t y sea T_h el árbol resultante de particionar t en $t_{h \leq}$ y $t_{h >}$ mediante el hiperplano h . Se escogería, por lo tanto, el hiperplano tal que:

$$h = \arg \max_h (e(T) - e(T_h)) \quad (8)$$

No obstante, el uso del error de clasificación para los criterios de selección y partición está desaconsejado, y es mejor usar otros criterios relacionados con la impureza del árbol, como la anteriormente mencionada entropía. El criterio más habitual es el llamado Índice de Gini o I_G , definido a partir de la probabilidad $p_i(t)$ de cada clase i en una hoja t como:

$$I_G(t) = \sum_{i=1}^k p_i(t)(1 - p_i(t)) \quad (9)$$

Entonces, para todos los hiperplanos posibles h que particionan la hoja t , se escogería aquel que maximiza la reducción en impureza:

$$h = \arg \max_h (I_G(t) - I_G(t_h)) \quad (10)$$

Existen otros criterios para decidir cómo particionar una hoja. El lector interesado puede referirse al trabajo de Rokach y Maimon (2005).

Selección del hiperplano

La selección del hiperplano o corte se realiza por fuerza bruta para cada una de las variables, atendiendo a su tipo. Si se trata de una variable numérica o una variable categórica ordinal, lo que se hace es ordenar el conjunto de datos de acuerdo a dicha variable a_j y recorrer todos los posibles hiperplanos h , seleccionando aquel corte $a_j \leq h$, es decir, variando los valores de h entre el mínimo y el máximo encontrados al ordenar todos los valores posibles. Este proceso es costoso para conjuntos de entrenamiento grandes, pero conforme se va construyendo el árbol, el número de hiperplanos a comprobar se va reduciendo considerablemente.

En el caso de una variable categórica no ordinal, donde no existe un orden implícito entre los posibles valores que puede tomar, es necesario generar todos los subconjuntos posibles, de forma que el hiperplano es en realidad una comparación de si un valor pertenece a un subconjunto o no. El número de subconjuntos posibles (sin incluir el subconjunto vacío o el subconjunto tri-

Lectura complementaria

L. Rokach, O. Maimon (2005). «Top-down induction of decision trees classifiers: A survey». IEEE Transactions on Systems, Man and Cybernetics Part C: Applications and Reviews, Vol. 35(4) pp. 476-487

vial conteniendo todos los elementos) para una variable categórica que toma c valores diferentes es $2^c - 2$, y el número de combinaciones que es necesario probar es de $2^{c-1} - 1$ (la mitad), número que crece exponencialmente con c .

3.2 Poda del árbol

Uno de los principales problemas de los árboles de decisión es que están orientados a obtener una partición del conjunto de entrenamiento «completa», es decir, intentan clasificar correctamente todos los elementos del conjunto de entrenamiento, aun cuando ello implique hacer crecer el árbol hasta profundidades muy elevadas, creando árboles enormes con cientos o miles (o incluso millones) de nodos. El problema es que dicho árbol es muy específico para el conjunto de entrenamiento, y seguramente cometerá muchos errores para datos nuevos, causando sobreentrenamiento. Es como si se diseñara un mueble para almacenar una colección muy específica de objetos; si se presenta un objeto nuevo nunca visto anteriormente, posiblemente no encajará bien en ninguna parte, ya que el mueble es demasiado específico para la colección para la que fue construido.

Para resolver este problema existe una segunda fase en el proceso de creación de un árbol de decisión llamada poda, dado que consiste en eliminar aquellas hojas que son las que causan el sobreentrenamiento. Para ello se calcula cuál es la partición (de la que cuelgan las hojas a eliminar o, en general, un subárbol) que aporta menor ratio entre el incremento de profundidad media del árbol y el decremento del error global de clasificación. Es decir, se eliminan aquellas hojas (o subárboles) que menos ayudan a mejorar el árbol durante el proceso de creación.

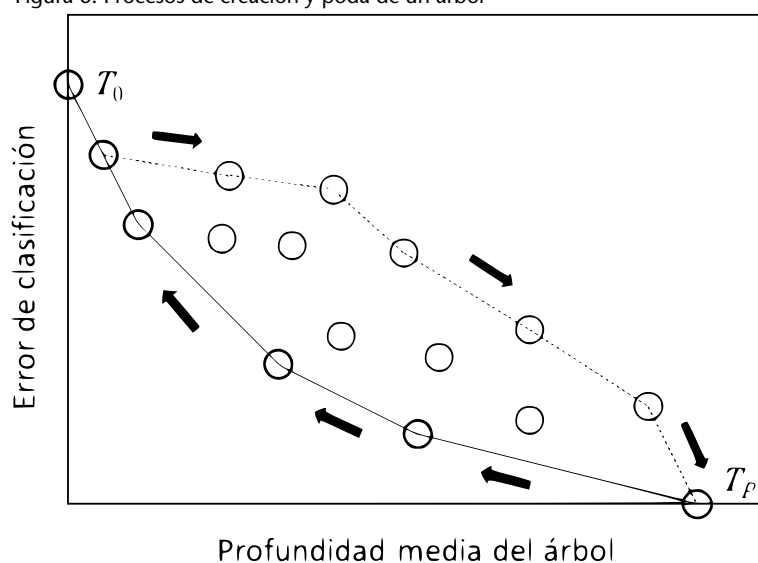
Desafortunadamente, esto no puede ser realizado durante el propio proceso de creación, ya que exigiría comprobar todos los subárboles existentes, número que crece exponencialmente a cada paso del algoritmo de crecimiento. Sin embargo, una vez se ha alcanzado el árbol puro que clasifica perfectamente el conjunto de entrenamiento, sí que es posible encontrar de forma eficiente la secuencia de subárboles óptimos, que son aquellos que proporcionan el menor error de clasificación en el conjunto de entrenamiento para cada una de las posibles profundidades medias.

Sea n_t el número de elementos de una hoja t y n el número total de elementos del conjunto de datos de entrenamiento. Sea $l(t)$ la profundidad de una hoja t . Entonces, definimos la profundidad media $l(T)$ de un árbol T como:

$$l(T) = \frac{1}{n} \sum_{t \in T} n_t l(t) \quad (11)$$

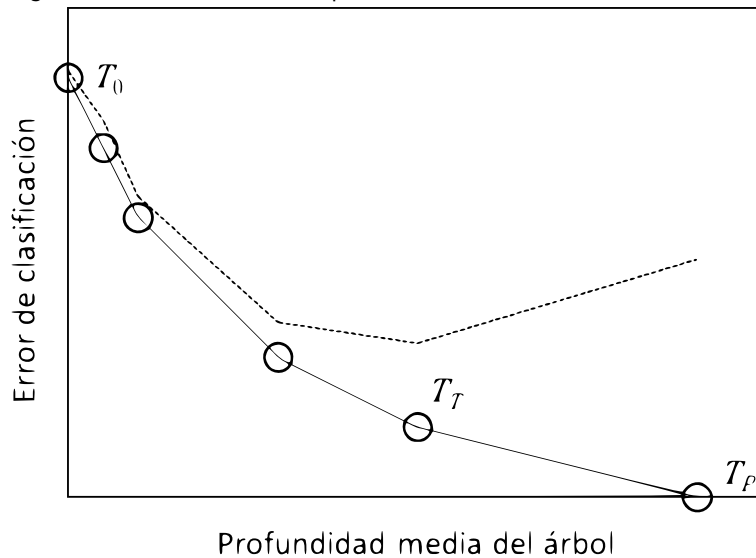
La figura 6 muestra la idea detrás del algoritmo de poda. Cada nodo o circunferencia representa un posible árbol, siendo T_0 el árbol inicial constituido por una sola hoja que contiene todos los elementos del conjunto de entrenamiento. Durante el proceso de creación del árbol, indicado por las flechas grises, se toman decisiones que generan una secuencia de árboles, unidos por la línea de puntos, los cuales aumentan la profundidad media y van reduciendo el error de clasificación. Si otras hojas hubieran sido seleccionadas para ser particionadas durante dicho proceso, se hubiera generado otra secuencia de árboles posibles, representados por los nodos en gris. Cuando se alcanza el árbol completo T_P , el algoritmo de poda selecciona, indicado por las flechas negras, aquellos subárboles que son óptimos, es decir, aquellos que minimizan el error de clasificación para una profundidad media dada.

Figura 6. Procesos de creación y poda de un árbol



Es entonces cuando entra en juego el conjunto de datos de test no usado para crear el árbol de decisión. Para cada uno de los subárboles óptimos encontrados por el proceso de poda, se mide el error de clasificación cometido en el conjunto de datos de test. Empezando por T_0 , lo más habitual es que conforme crece $l(T)$ para cada uno de los subárboles óptimos encontrados por el algoritmo de poda, el error cometido en el conjunto de test (indicado por la línea de puntos) también vaya descendiendo, como sucede con el conjunto de entrenamiento. Sin embargo, es posible que, a partir de cierto punto, el error en el conjunto de test se estabilice o incluso repunte, indicando que árboles más grandes son demasiado específicos y sufren de sobreentrenamiento. Así, tal como muestra la figura 7, el resultado del proceso de creación de un árbol de decisión sería aquel subárbol T_T que minimiza el error cometido en el conjunto de test.

Figura 7. Selección del subárbol óptimo



Se recuerda al lector que en el material adicional que acompaña este libro puede encontrar ejemplos de creación y poda de árboles de decisión usando jupyter y R.

3.3 Resumen

Existen muchas razones por las cuales los árboles de decisión son uno de los modelos más utilizados en la práctica:

- Su construcción es sencilla, aunque puede ser costosa computacionalmente para conjuntos de entrenamiento muy grandes.
- El resultado obtenido es directamente interpretable, y también es posible evaluar la importancia de cada variable utilizada en la construcción del modelo.
- Pueden combinar variables numéricas y categóricas en el mismo modelo, siendo invariantes a traslaciones y escalados de los datos, por lo que no es necesario normalizar los datos (aun cuando esto sea recomendable en general).
- Pueden trabajar con valores perdidos, utilizando condiciones alternativas (conocidas como *surrogate splits*).
- Su implementación práctica se reduce a una serie de reglas que pueden ser fácilmente escritas como un conjunto de sentencias «if-then-else».

Por otra parte, los árboles de decisión presentan una serie de problemas que, aun siendo conocidos, no son sencillos de resolver sin una inspección del árbol construido (por otra parte, siempre necesaria):

- **Fragmentación:** a veces una hoja es particionada en dos de forma muy desequilibrada, generando dos nuevas hojas, una con pocos elementos y la otra con la mayoría. Este proceso puede generar ramas muy largas, incrementando la profundidad media del árbol y fragmentando el espacio de datos de entrada en regiones muy pequeñas, seguramente no representativas. La única solución consiste en imponer unos requisitos mínimos a la hora de particionar una hoja, de forma que las dos nuevas hojas estén equilibradas.
- **Repetición:** este fenómeno aparece cuando la secuencia de condiciones tomada en cada nodo interno hasta llegar a una hoja repite la misma variable, cambiando el valor por el cual se toma la decisión. Esto indica que dicha variable tiene una relación claramente no lineal con respecto a la variable objetivo, por lo que se puede intentar realizar una transformación no lineal de dicha variable a partir de un análisis de su distribución.
- **Replicación:** en muchos casos, los datos de entrada pueden presentar una estructura interna que es imposible de capturar si se usan hiperplanos o condiciones ortogonales, es decir, que solo utilizan una variable en cada decisión o nodo interno. Lo que suele ocurrir es que los subárboles generados reproducen la estructura de variables, indicando dicho problema. La única solución posible es introducir el uso de hiperplanos oblicuos o bien realizar algún procedimiento de extracción de características en los datos originales, de forma que se reduzca la colinealidad entre variables y se elimine dicha estructura interna, capturándola en una nueva variable.

Los árboles de decisión son muy sensibles al conjunto de datos de entrenamiento, por lo que la presencia de *outliers* puede generar ramas profundas que no generalicen bien para nuevos datos. Aunque el proceso de poda seguramente eliminará dichas ramas, es mejor construir el árbol de decisión una vez se han detectado y eliminado los *outliers* del conjunto de entrenamiento.

4. Máquinas de soporte vectorial

Las máquinas de soporte vectorial (en inglés, *Support Vector Machines* o SVM) es el nombre con el que se conoce un algoritmo de aprendizaje supervisado capaz de resolver problemas de clasificación, tanto lineales como no lineales. Actualmente está considerado como uno de los algoritmos más potentes en reconocimiento de patrones.

Su eficiencia y los buenos resultados obtenidos en comparación con otros algoritmos han convertido esta técnica en la más utilizada en campos como el reconocimiento de textos y habla, predicción de series temporales y estudios sobre bases de datos de marketing, entre otros, pero también en otros campos como la secuenciación de proteínas y el diagnóstico de varios tipos de cáncer.

La gran aportación de Vapnik (Vapnik y Chervonenkis, 1974) radica en que construye un método que tiene por objetivo producir predicciones en las que se puede tener mucha confianza, en lugar de lo que se ha hecho tradicionalmente, que consiste en construir hipótesis que cometan pocos errores.

La hipótesis tradicional se basa en lo que se conoce como minimización del riesgo empírico (*empirical risk minimization*), mientras que el enfoque de las SVM se basa en la minimización del riesgo estructural (*structural risk minimization*), de modo que lo que se busca es construir modelos que estructuralmente tengan poco riesgo de cometer errores ante clasificaciones futuras. El concepto de minimización del riesgo estructural fue introducido en 1974 por Vapnik y Chervonenkis (Vapnik y Chervonenkis, 1974).

Para entender mejor tanto la teoría de Vapnik y Chervonenkis aplicada a las SVM, vamos a simplificar y focalizarnos en el problema de la clasificación binaria, en la que a partir del conjunto de datos de entrenamiento se construye un hiperplano (separador lineal) capaz de dividir los puntos en dos grupos.

La idea detrás de las máquinas de soporte vectorial es muy intuitiva. Si la frontera definida entre dos regiones con elementos de clases diferentes es compleja, en lugar de construir un clasificador complejo que reproduzca dicha frontera, lo que se intenta es «doblar» el espacio de datos en un espacio de mayor dimensionalidad, de forma que con un único corte (es decir, un clasificador muy sencillo) se puedan separar fácilmente ambas regiones.

Lectura complementaria

V. N. Vapnik, A. Ya. Chervonenkis, «On the method of ordered risk minimization. I», *Avtomat. i Telemekh.*, 1974, no. 8, 21-30; *Autom. Remote Control*, 35:8 (1974), pp. 1226-1235

4.1 Definición de margen e hiperplano separador

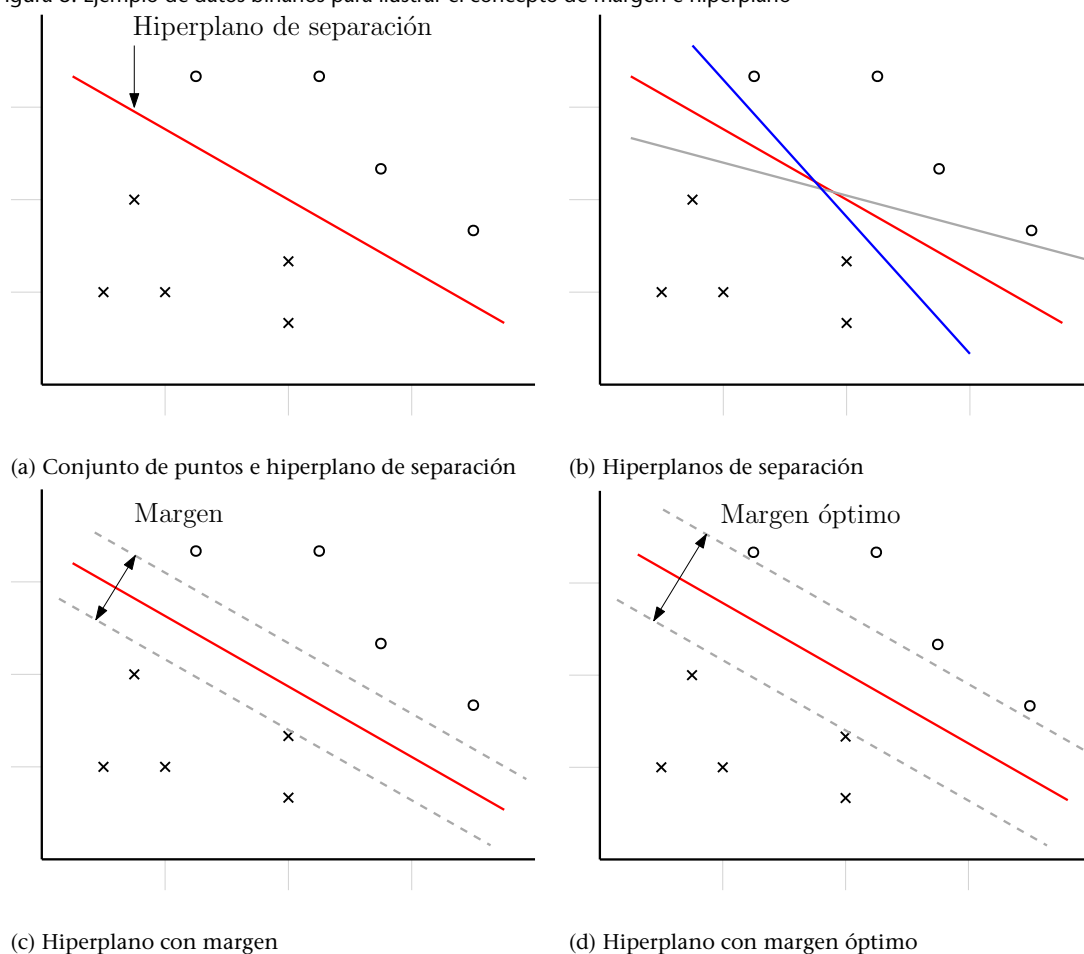
Definiremos el **margen** como la zona de separación entre los distintos grupos a clasificar. Esta zona de separación quedará delimitada a través de **hiperplanos**.

Uno de los factores diferenciadores de este algoritmo respecto de otros clasificadores es su gestión del concepto de margen. Las SVM buscan maximizar el margen entre los puntos pertenecientes a los distintos grupos a clasificar. Maximizar el margen implica que el hiperplano de decisión o separación esté diseñado de tal forma que el máximo número de futuros puntos queden bien clasificados. Por este motivo, como veremos más adelante, las SVM utilizan las técnicas de optimización cuadrática propuestas por el matemático italiano Giuseppe Luigi Lagrange.

El objetivo de las SVM es encontrar el hiperplano óptimo que maximiza el margen entre clases (variable objetivo) del juego de datos de entrenamiento.

Veamos en la figura 8a un gráfico de dispersión del juego de datos de entrenamiento, donde vemos las dos clases presentes en el conjunto de datos.

Figura 8. Ejemplo de datos binarios para ilustrar el concepto de margen e hiperplano



A partir de la figura 8a podemos observar que a partir de un hiperplano podríamos separar las dos clases de datos de forma clara. Diremos que esta recta de separación es en realidad un **hiperplano de separación**.

Debemos tener en cuenta que el hecho de disponer de un hiperplano separador no nos garantiza, en absoluto, que este sea el mejor de todos los posibles hiperplanos separadores. Vemos en la figura 8b que en nuestro ejemplo podemos tener infinitos hiperplanos separadores, todos ellos perfectamente válidos para separar nuestro juego de datos de clientes entre aquellos que contratan y aquellos que no contratan el servicio de seguro.

Ante la pregunta de si hay o no un hiperplano separador mejor que otro, pensemos, por ejemplo, que aquellos hiperplanos que estén demasiado cerca de los puntos del gráfico muy probablemente no serán demasiado buenos para clasificar nuevos puntos, ya que correremos mucho riesgo de acabar teniendo puntos en el lado no deseado del hiperplano. Por lo tanto, nuestra intuición nos dice que sería conveniente fijar un margen de seguridad capaz de establecer una distancia entre los puntos de cada clase de puntos a clasificar.

Como puede observarse en la figura 8c, el margen es un «terreno de nadie», donde no deberíamos encontrar ningún punto del juego de datos de entrenamiento. Para construir el margen asociado a un hiperplano, procederemos a calcular la distancia entre el hiperplano y el punto más cercano a este. Una vez tenemos esta distancia, la doblamos y este será nuestro margen. De este modo, cuanto más cerca esté un hiperplano de los puntos, menor será su margen. Por contra, cuanto más alejado esté un hiperplano de los puntos, mayor será su margen y, en consecuencia, menor será su riesgo de clasificar incorrectamente nuevos puntos del juego de datos.

Así, el **hiperplano separador óptimo** se define como el hiperplano que tenga mayor margen posible, tal y como podemos ver en la figura 8d. El objetivo de las SVM es encontrar el hiperplano separador óptimo que maximice el margen del juego de datos de entrenamiento.

4.2 Cálculo del margen y del hiperplano separador óptimo

En este subapartado usaremos cálculo vectorial para poder operar con vectores y estudiar las propiedades de los mismos, que nos van a permitir obtener la ecuación del hiperplano separador con el margen óptimo.

4.2.1 La ecuación del hiperplano

La expresión más habitual para representar una recta en el plano es $y = ax + b$. Si generalizamos este caso a un espacio de más dimensiones, por cuestiones

de practicidad se suele usar la expresión $w^T \cdot x = 0$, donde la equivalencia con la expresión anterior se muestra a continuación:

$$w = \begin{pmatrix} -b \\ -a \\ 1 \end{pmatrix}, x = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix} \quad (12)$$

A partir de la notación, es fácil operar y ver que estas dos expresiones son equivalentes:

$$w^T \cdot x = (-b)(1) + (-a)x + (1)y = y - ax - b \quad (13)$$

Aunque ambas ecuaciones son equivalentes, generalmente, es más cómodo trabajar con la expresión $w^T \cdot x$ por los siguientes motivos:

- Es más adecuada para más de dos dimensiones.
- El vector w siempre será perpendicular al hiperplano, por definición.

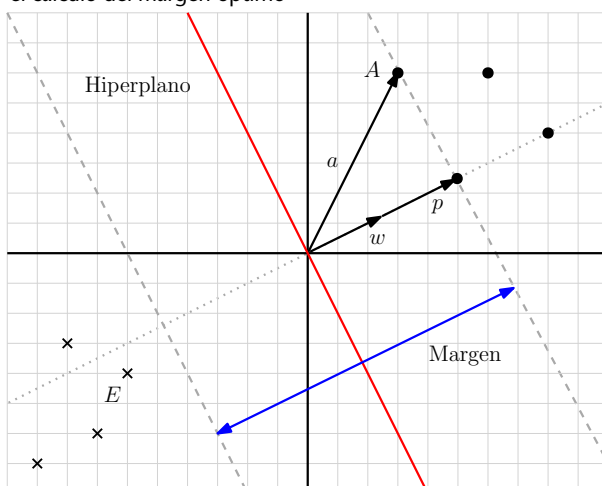
Un cálculo que se repite constantemente en las SVM es la distancia entre un punto cualquiera x_0 (según la notación general se trataría de cualquier punto d_i de los que forman el conjunto de datos D) y un hiperplano definido por su vector normal w . Este cálculo se efectúa mediante una proyección del vector definido entre el punto x_0 y el hiperplano en cuestión sobre el vector normal al hiperplano. De forma resumida:

$$d = \frac{|x_0 \cdot w|}{\|w\|} \quad (14)$$

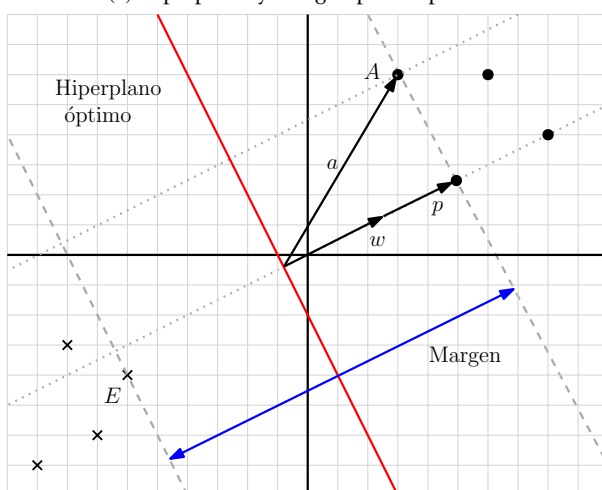
Entonces, si x_0 era el punto más cercano al hiperplano, podemos definir el margen como el doble de la distancia calculada (extendiendo dicha distancia a cada lado del hiperplano), dado que en ese margen no existe ningún otro punto.

En la figura 9a podemos apreciar como el margen calculado para el punto A no es el óptimo, ya que podríamos conseguir un margen mayor hasta alcanzar el punto E, haciendo retroceder el hiperplano en cuestión. Sin embargo, en la figura 9b vemos como el margen alcanza los puntos A y E que marcan la frontera entre los dos grupos de puntos. Diremos entonces que el margen e hiperplano generados son los óptimos.

Figura 9. Ejemplo de datos bidimensionales para ilustrar el cálculo del margen óptimo



(a) Hiperplano y margen para el punto A



(b) Hiperplano óptimo entre los puntos A y E

Más adelante veremos como el algoritmo SVM llama a los puntos A y E, **vectores de soporte**, es decir, vectores definidos por puntos que marcarán la frontera que nos servirá para calcular el margen y el hiperplano separador.

Obviamente, los conceptos hiperplano y margen están intrínsecamente relacionados, puesto que fijado un margen podemos obtener el hiperplano que pasa justo por su centro y, al revés, fijado un hiperplano podemos obtener el margen a partir de la distancia de los puntos frontera con el hiperplano. Por lo tanto, podemos concluir que determinar el mayor margen posible es equivalente a encontrar el hiperplano óptimo.

El proceso de optimización se completa con el método de los **multiplicadores de Lagrange**, adecuado para encontrar máximos y mínimos de funciones continuas de múltiples variables y sujetas a restricciones. En dos dimensiones, los multiplicadores de Lagrange permiten resolver el problema de maximizar una función $f(x,y)$ sujeta a las restricciones $g(x,y) = 0$, con la única condición de que ambas tengan derivadas parciales continuas, es decir, que su forma sea

«suave» en todas sus variables. De este modo, Lagrange optimizará la función $f(x,y) - \lambda \cdot g(x,y)$, donde λ es el coste de la restricción. Esto es generalizable a cualquier número de dimensiones.

En el caso de las SVM, el objetivo es encontrar aquel hiperplano que maximiza el margen (la distancia del punto más cercano a dicho hiperplano) y que satisface que todos los puntos de una clase están a un lado del hiperplano y todos los de la otra clase están al otro lado. Cuando esto es posible se habla de «hard-margin». Si esto no es posible (es decir, el conjunto de datos no es linealmente separable), se habla de «soft-margin» y es necesario añadir una condición más de forma que el hiperplano óptimo minimice el número de elementos de cada clase que se encuentran en el lado equivocado, teniendo en cuenta también su distancia.

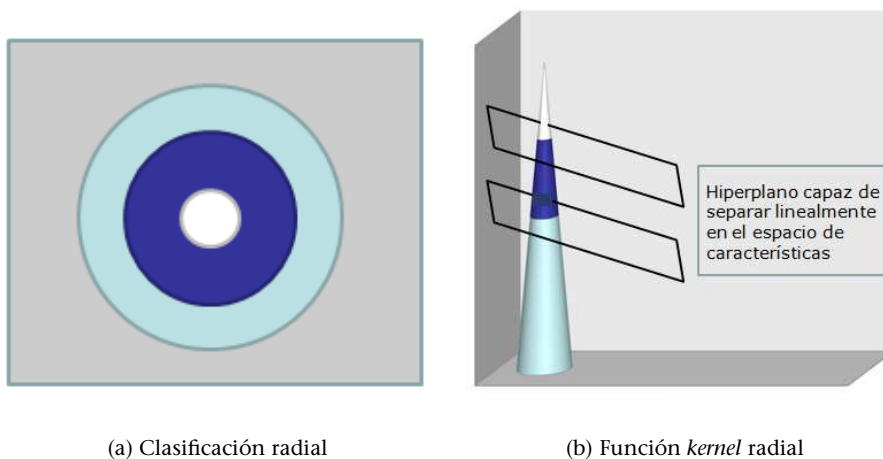
4.3 Funciones *kernel*

Estrictamente hablando, las SVM se ocupan tan solo de resolver problemas de clasificación lineal y se apoyan en las funciones *kernel* para transformar un problema no lineal en el espacio original X en un problema lineal en el espacio transformado F . Las funciones *kernel* son las que «doblan» el espacio de entrada para poder realizar cortes complejos con un simple hiperplano.

4.3.1 ¿Cómo pueden ayudarnos las funciones *kernel*?

La figura 10a nos muestra un problema de clasificación con tres zonas de distintos colores. Claramente se trata de un problema de clasificación no lineal, puesto que visualmente observamos cómo solo dos circunferencias concéntricas son capaces de realizar una correcta clasificación.

Figura 10. Ejemplo de clasificación y función *kernel* radial



Supongamos que disponemos de una función *kernel* capaz de transportar la figura anterior a un espacio de tres dimensiones, tal y como podemos ver en

la figura 10b. De hecho, podemos pensar la figura 10a como un plano en picado de un cono dibujado, tal y como vemos en la figura 10b.

En el espacio transformado por la función *kernel* (también llamado espacio de características) sí que podemos separar los tres colores por planos, es decir, tenemos un problema de clasificación lineal. En este nuevo espacio podríamos identificar el margen óptimo de separación para clasificar las tres zonas de colores diferentes.

4.3.2 Función *kernel*

Una función *kernel* es aquella que a cada par de vectores de un espacio origen $X \subseteq \mathbb{R}^n$, asigna un número real, cumpliendo con las propiedades del producto escalar.

$$k : X \times X \rightarrow \mathbb{R}, \text{ donde } X \subseteq \mathbb{R}^n \quad (15)$$

Las propiedades que debe cumplir una función *kernel* son:

- La propiedad distributiva.
- La propiedad conmutativa.
- La propiedad semidefinida positiva.

En términos algebraicos, el producto escalar es la suma de los productos de los correspondientes componentes de cada vector. Así mismo, en términos geométricos podemos pensar el producto escalar como la proyección de un vector sobre otro, tal y como hemos estudiado en el subapartado anterior.

En definitiva, una función *kernel* no es más que una versión «modificada» de la definición clásica de producto escalar. Este tipo de funciones reciben el nombre de funciones *kernel* porque se construyen a partir de una asignación de pesos a los distintos componentes de cada vector, tal y como podemos apreciar en la ecuación 16.

4.3.3 Algoritmo SVM

Para entenderlo mejor, veamos un caso de clasificación binaria, es decir, cada punto de nuestro espacio de características, $x_i \in X$, estará asociado a una clase binaria, i.e. $c_i \in \{-1, 1\}$.

Podríamos pensarlo como la medida de similitud que nos permita determinar si un nuevo punto se encuentra a la derecha o a la izquierda de un hiperplano de separación.

En este caso, nuestra función de clasificación podría consistir en calcular la suma de distancias alteradas por un peso w , como se muestra a continuación:

$$h(z) = \text{signo}\left(\sum_{i=1}^n w_i \cdot c_i \cdot k(x_i, z)\right) \quad (16)$$

donde:

- n es el número de entradas del juego de datos de entrenamiento.
- z es el nuevo punto a clasificar.
- $h(z) \in \{-1, 1\}$ es la función de clasificación.
- $k : X \times X \rightarrow \mathbb{R}$ es la función *kernel* que mide la similitud entre puntos (producto escalar, distancia, etc).
- El conjunto de pares $\{(x_i, c_i)\}_{i=1}^n$ son los puntos etiquetados del juego de datos. Es decir, constituyen el juego de datos de entrenamiento, donde $c_i \in \{-1, 1\}$ es la etiqueta o clase del punto x_i .
- $w_i \in \mathbb{R}$ son los pesos que el algoritmo ha determinado para el juego de datos de entrenamiento.
- $\text{signo}()$ es la función que nos devuelve el signo de un número, i.e. + o -.

El trabajo del algoritmo SVM será, precisamente, determinar los valores w óptimos en términos de maximización del margen, con el objetivo de encontrar el mejor hiperplano de separación.

4.3.4 El método *kernel*

En ocasiones puede ser interesante añadir a la ecuación 15 una función adicional de transformación del espacio original X en un nuevo espacio intermedio V :

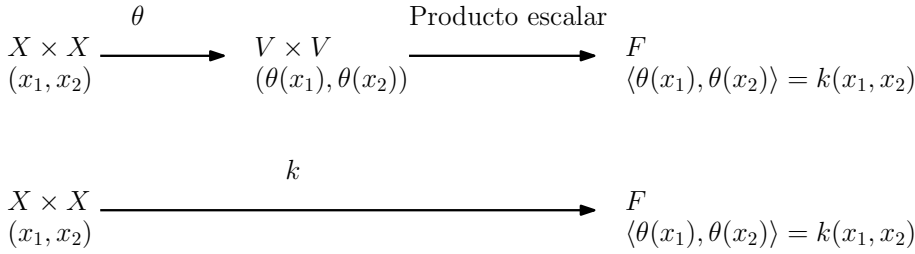
$$\phi : X \rightarrow V \text{ donde } X \subseteq \mathbb{R}^n; V \subseteq \mathbb{R}^m \quad (17)$$

con $m > n$, de tal modo que:

$$k : X \times X \rightarrow V \times V \rightarrow F \text{ donde } X \subseteq \mathbb{R}^n; V \subseteq \mathbb{R}^m; F \subseteq \mathbb{R} \quad (18)$$

La figura 11 muestra el detalle del proceso comentado hasta ahora, donde nuevamente la operación $\langle , \rangle$ deberá cumplir la condición de ser producto escalar.

Figura 11. Función *kernel* como producto escalar



De este modo, nuestra función *kernel* $k(x_1, x_2)$ podría pensarse como un producto escalar $\langle \phi(x_1), \phi(x_2) \rangle$, mucho más fácil de operar, ya que podemos aplicar las propiedades algebraicas y geométricas del producto escalar.

Esta transformación del espacio de características, que consiste en hacer que nuestra función *kernel* opere en un espacio de más dimensiones, podréis encontrarla en la literatura inglesa como *kernel trick* o método *kernel*.

Gracias a este tipo de transformaciones, conseguimos que juegos de datos que en el espacio $\mathbb{R}^n$ no son linealmente separables, sí que lo sean en un espacio de mayor dimensionalidad, i.e. $\mathbb{R}^m$ donde $m > n$.

La principal potencia de este método es que en realidad no necesitamos trabajar explícitamente en el espacio $\mathbb{R}^m$, ya que simplemente necesitamos operar el producto escalar del espacio transformado. A continuación veremos algunos ejemplos de transformaciones.

Transformación cuadrática en un espacio de más dimensiones

La transformación:

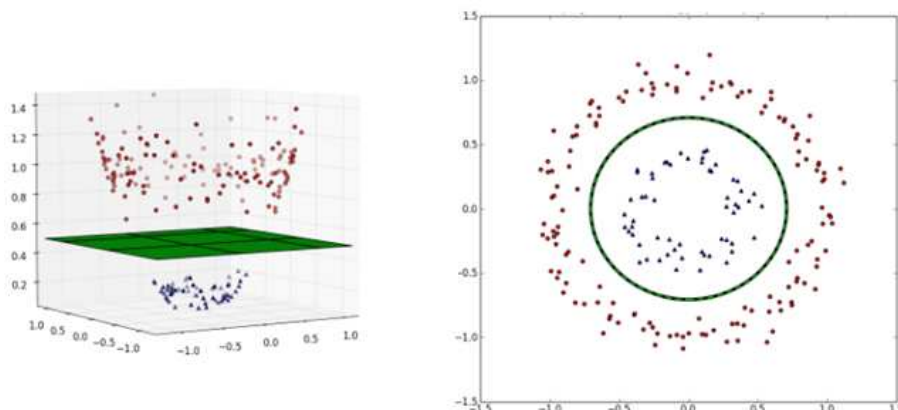
$$[u_1, u_2] = [u_1, u_2, u_1^2 + u_2^2] \quad (19)$$

es la función $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ que usaremos para una función *kernel* lineal (producto escalar clásico), de modo que:

$$\begin{aligned}
 k(u, v) &= u^T \cdot v = \langle \phi(u_1, u_2), \phi(v_1, v_2) \rangle \\
 &= \langle (u_1, u_2, u_1^2 + u_2^2), (v_1, v_2, v_1^2 + v_2^2) \rangle
 \end{aligned} \quad (20)$$

En la figura 12 vemos muy gráficamente el beneficio de nuestra transformación, donde añadiendo adecuadamente una dimensión más conseguimos trabajar con un juego de datos que sí es linealmente separable.

Figura 12. Ejemplo de *kernel trick*



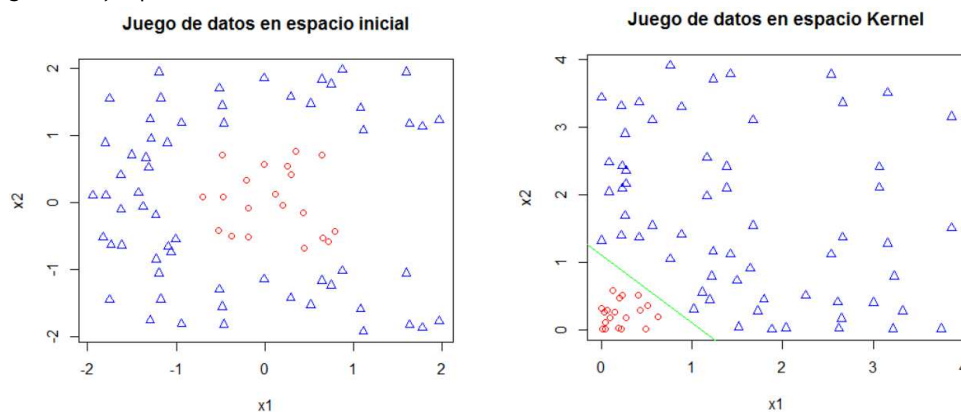
Transformación cuadrática manteniendo las dimensiones

Imaginemos una distribución concéntrica como la que apreciamos en la figura 13a.

Lo que quisiéramos es una función que transforme el espacio original en un espacio de características en el que la distribución de puntos sea linealmente separable, es decir, que sea separable a través de un hiperplano.

Una función *kernel* que nos ayudará a realizar esta transformación es la función cuadrática que simplemente consiste en considerar el cuadrado de cada uno de los dos componentes de cada punto, manteniendo el número de dimensiones. El espacio de características que conseguiríamos sería el que apreciamos en la figura 13b.

Figura 13. Ejemplo de transformación cuadrática manteniendo las dimensiones



(a) Distribución de puntos en el espacio original (b) Distribución de puntos en el espacio de características

La función *kernel* de nuestro ejemplo podría tener dos expresiones equivalentes:

$$k(u,v) = u_1^2 v_1^2 + u_2^2 v_2^2 \quad (21)$$

donde $u,v \in X \subseteq \mathbb{R}^2$, y

$$k(u,v) = \langle (u_1^2, u_2^2), (v_1^2, v_2^2) \rangle = \langle \phi(u), \phi(v) \rangle \quad (22)$$

donde $u,v \in X \subseteq \mathbb{R}^2$.

Probablemente, las matemáticas nos ayudan más si lo pensamos como la ecuación 22, dado que a partir de la definición de producto escalar podemos encontrar el margen óptimo de separación.

Transformación polinomial

La transformación:

$$[u_1, u_2] = [u_1^2, u_1 \cdot u_2 \sqrt{2}, u_1 \cdot \sqrt{2b}, u_2 \cdot \sqrt{2b}, b] \quad (23)$$

es la función $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^5$ que usaremos para una función *kernel* polinomial

$$k(u,v) = (u^T \cdot v + b)^2 \quad (24)$$

Esta técnica es muy eficiente en términos de computación porque en realidad siempre estamos computando un producto escalar. En un espacio con más dimensiones, pero al final siempre computamos un producto escalar para medir la distancia de nuestro punto o vector al hiperplano de separación.

4.4 Tipos de funciones *kernel*

Una de las tareas que realiza el algoritmo SVM es identificar los **vectores de soporte**, es decir, aquellos vectores que marcarán la trayectoria del hiperplano separador.

Los vectores de soporte marcan la frontera. Sería como una línea de seguridad que una vez sobrepasada deja de ofrecer garantías de fiabilidad en la predicción, de modo que a la hora de valorar un modelo predictivo basado en las SVM podríamos plantearnos el cuantificar, por ejemplo, qué porcentaje de predicciones se encuentran alejadas de los vectores de soporte y qué porcentaje de predicciones se encuentran en sus alrededores.

Veremos a continuación que cada estrategia de clasificación (lineal, polinomial, radial y sigmoidal) tiene su propia función *kernel*. Tal y como menciona el autor Jean Philippe Vert, la idea que hay detrás de todas ellas es un concepto equivalente al del producto escalar que persigue medir el grado de similitud entre dos puntos (vistos como vectores) de nuestro juego de datos.

Lectura complementaria

B. Schölkopf, K. Tsuda, J.-P. Vert. «Kernel Methods in Computational Biology». MIT Press, 2004.

4.4.1 Kernel lineal

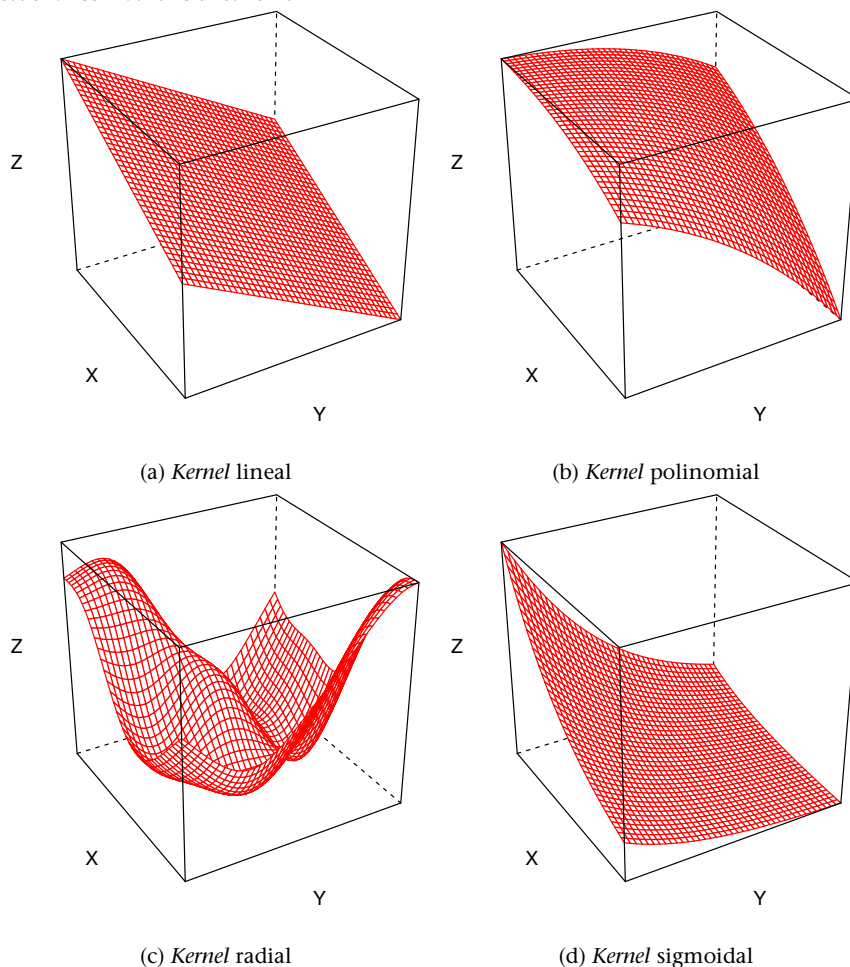
La función *kernel* utilizada para la SVM de clasificación lineal es:

$$k(u,v) = u^T \cdot v \quad (25)$$

donde $u, v \in X$.

Observamos que simplemente se trata del producto escalar de dos vectores y no supone ninguna transformación del espacio de características. En este caso, la función *kernel* responde enteramente a lo que nuestra intuición entiende por producto escalar. La figura 14a muestra un ejemplo de la superficie de predicción creada por un modelo lineal.

Figura 14. Ejemplo de las distintas formas en el espacio transformado que podemos gestionar con las funciones *kernel*



4.4.2 Kernel polinomial

La función *kernel* polinomial es:

$$k(u,v) = (\gamma u^T \cdot v + b)^p \quad (26)$$

donde $u, v \in X$ y $\gamma > 0$, $b \geq 0$ y $p > 0$ son parámetros.

Podríamos pensar el *kernel* polinomial como una primera generalización del concepto de producto escalar, en el que tenemos la opción de trabajar en un espacio transformado de más dimensiones que el espacio original.

Los parámetros γ, b, p nos permiten modelar en cierto grado la morfología de la transformación del espacio original. El calibrage de estos parámetros y la influencia que cada uno de ellos ejerce sobre la transformación del juego de datos, forman parte del estudio necesario para poder usar de forma eficiente esta función *kernel*. La figura 14b muestra un ejemplo de la superficie de predicción creada por un modelo polinomial.

4.4.3 Kernel radial

La función *kernel* radial es:

$$k(u,v) = e^{-\gamma \|u-v\|^2} \quad (27)$$

donde $u, v \in X$ y $\gamma > 0$ es parámetro.

También conocido como *kernel* de Gauss, generaliza el concepto de distancia entre vectores en lugar de su producto escalar. Está especialmente indicada para modelar relaciones complejas y no lineales, como podemos ver en el ejemplo de superficie de predicción de la figura 14c. Esta función es muy dependiente del parámetro γ , de modo que será importante tenerlo en cuenta en aspectos como el posible sobreentrenamiento del juego de datos.

4.4.4 Kernel sigmoidal

La función *kernel* sigmoidal es:

$$k(u,v) = \tanh(\gamma u^T \cdot v + b) \quad (28)$$

donde $u, v \in X$ y $\gamma > 0$, $b \geq 0$ son parámetros.

Se trata de la tangente hiperbólica del producto escalar en el espacio original, de modo que sin duda se trata de otro caso de generalización del producto escalar.

Es capaz de modelar relaciones complejas similares al *kernel* radial. La tangente hiperbólica es una función muy utilizada como función de activación en las redes neuronales y por este motivo también tiene un papel importante en el mundo de las SVM. La figura 14d muestra un ejemplo de la superficie de predicción creada por un modelo sigmoideal.

Sin embargo, tiene el riesgo que para ciertos valores de sus parámetros γ, b puede llegar a representar una función *kernel* no válida, es decir, que no cumpla con las condiciones que se le exigen a una función *kernel* para ser considerada como tal.

4.5 Resumen

La apuesta de las máquinas de soporte vectorial por maximizar las opciones de clasificar correctamente las nuevas entradas del juego de datos convierte este algoritmo en un clasificador muy robusto cuando los datos son claramente separables (mediante un hiperplano); sin embargo, cuando hay ruido, sobre todo en la franja de separación, puede dar algunos problemas.

Las SVM son muy efectivas en juegos de datos con muchas dimensiones o atributos, sacando provecho de sus propiedades geométricas. Además, gracias al recurso de los vectores de soporte que funcionan como puntos de referencia, se consigue una gran eficiencia computacional.

Actualmente, las SVM son un algoritmo muy utilizado en tareas de clasificación de documentos o también en el ámbito de la salud, como por ejemplo en la prevención y detección de enfermedades de diagnóstico complejo.

En definitiva, y a pesar de que también pueden usarse para tareas de regresión, las máquinas de soporte vectorial son un algoritmo muy eficiente y ampliamente usado en tareas de clasificación. El uso que hacen de las funciones *kernel* y su visión puramente vectorial del problema de clasificación le confieren una potencia matemática que explica claramente su buen rendimiento ante otras tipologías de algoritmos.

El principal problema de las SVM es la dificultad para entender el hiperplano de corte en el espacio de mayor dimensión donde se realiza la separación de elementos de clases diferentes, dado que puede no resultar trivial entender el «pliegue» del espacio de entrada que realiza la función *kernel*, funcionando entonces como un modelo de caja negra.

Resumen

En este módulo didáctico hemos presentado las principales características del aprendizaje automático supervisado, así como algunos de los principales modelos supervisados.

Al inicio de este texto hemos presentado el algoritmo k -NN. Este es uno de los algoritmos más simples para realizar aprendizaje supervisado. Aun así, puede obtener resultados satisfactorios en algunos problemas y aplicaciones concretas.

A continuación, hemos revisado uno de los algoritmos más conocidos y empleados para tareas de aprendizaje supervisado, como son los árboles de decisión. Este tipo de modelo, aunque quizás una bondad en los resultados al nivel de otros modelos supervisados, permite un nivel de explicabilidad que lo hacen especialmente interesante en entornos donde es relevante poder interpretar la decisión que toma un modelo automático.

Finalmente, hemos introducido las máquinas de soporte vectorial, más conocidas como SVM, en el último apartado de este módulo didáctico, las cuales en muchos casos permiten obtener modelos con una precisión más elevada, aunque no permiten una fácil interpretación del modelo construido.

Glosario

función *kernel* *f* Funciones que permiten «doblar» el espacio m -dimensional de forma que permitan su manipulación (partirlo) mediante hiperplanos, capturando las relaciones no lineales presentes en los datos sin tener que transformarlos.

hiperplano *m* Elemento geométrico de dimensión inferior al espacio en el que se encuentra, de forma que divide dicho espacio en dos mitades. En el caso del espacio 2D, un hiperplano es una recta que separa el plano (2D) en dos partes, una a cada lado de la recta (hiperplano).

sobreajuste *m* El sobreajuste se produce cuando el algoritmo usado para crear un modelo captura en exceso la naturaleza intrínseca del conjunto de datos usado en el proceso de creación, imposibilitando que el modelo generalice bien para datos nunca observados con anterioridad, creando un sesgo.
en overfitting

Bibliografía

Cichosz, P. (2015). *Data mining algorithms. Explained using R*. Wiley.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition*. O'Reilly Media, Inc.

Gironés Roig, J.; Casas Roma, J.; Minguillón Alfonso, J.; Caihuelas Quiles, R. (2017). *Minería de datos: Modelos y algoritmos*. Barcelona: Editorial UOC.

